# Supplement to "MorletDB: Recordings, Not Rows"

Version 0.14

Zin Nam

## ABOUT THIS SUPPLEMENT

This supplement holds studies that support the system described in the main paper but are not needed for its argument: lossless storage of raw samples, envelope precision, building representations outside or inside the database, access control, recovery and load tests, and application evidence and negative results. Text, tables and numbers are carried over from draft 0.6 of the paper; protocols and raw results are in the repository [15]. Section and table references to "the paper" refer to the main manuscript.

The machine, PostgreSQL and pgvector versions are those of the main paper. TimescaleDB 2.30.2 ran in Docker; we compare its sizes, not its speed.

## 1 LOSSLESS CHUNK CODING

Each raw chunk is one `bytea` value. The codec combines established techniques: fixed polynomial predictors of order 0–4 as in Shorten and FLAC [2, 3], partitioned Rice coding of zig-zag mapped residuals [4], and a CRC-32C checksum. Its distinctive part is how it treats floats.

Most sensor values are calibrated integers: a digitizer emits counts $d$, and acquisition software stores $x = f(d)$, for example $x = d \cdot s$ (seismic counts, WAV) or, in EDF [5], $x = (d - d_{\min})(p_{\max} - p_{\min})/(d_{\max} - d_{\min}) + p_{\min}$. The recording declares a nominal quantum $q$: 1 for counts, 0.001 for three decimals, the ADC step for calibrated values. Per chunk, the encoder seeks a grid

$$x_i = o + k_i\, s \oplus \delta_i, \tag{1}$$

with integer indices $k_i$, a step $s$ refined from $q$ to within one unit in the last place (ulp), an offset $o$ (format version 3; zero in version 2), and corrections $\delta_i$, stored as differences of order-preserving integer images of float64, so that decoding reproduces every bit pattern. A division mode $x = k/d$ covers decimal steps exactly. Indices are predicted and Rice-coded like integers; corrections form a second stream. If no grid fits within a thousandth of a step, the chunk is coded as order-preserving float bits, and if that is not smaller, as raw float64. The quantum is a storage hint, never a conversion: a wrong quantum costs space, not accuracy.

## 2 STORAGE OF RAW SAMPLES

### 2.1 Database footprint (P2)

P2 appended the same 4,194,304 samples of four inputs (CX seismic counts [11], IMS bearing vibration [14], Paderborn bearing vibration [13] as ADC counts and as physical floats) to MorletDB and to TimescaleDB hypertables with native compression (Gorilla for floats; delta-of-delta and Simple-8b for integers), and measured total on-disk bytes, including indexes, TOAST and metadata tables, after verifying that four intervals per layout read back bit-exactly [16].

TimescaleDB on integer columns comes closest but is 24–35% larger (Table 1), and requires the user to convert values to counts first. On float columns, Gorilla's XOR coding *expands* noisy decimal and calibrated values beyond 64 bits per sample. We tested one `segmentby`/`orderby` configuration and not TimescaleDB's newer column formats. Of MorletDB's footprint, the codec payload is 4.6–9.2 bits per sample (Table 2); PostgreSQL row and TOAST overhead adds 1.5–2.6 bits per sample at 4096-sample chunks.

**Table 1: P2: total database storage in bits per sample. T: the TimescaleDB layouts tested (a row per sample with an integer or float column; a row per 4096-sample array).**

| Input | MorletDB | T int | T float | T array |
|---|---|---|---|---|
| CX seismic counts | **6.12** | 8.25 | 14.34 | 25.0 |
| IMS three-decimal | **11.36** | 14.06 | 71.95 | 28.1 |
| Paderborn counts | **9.95** | 12.89 | 24.58 | 26.4 |
| Paderborn physical | **11.53** | — | 70.12 | 29.5 |

**Table 2: B1: bits per sample. Z: zstd-19; PQ: Parquet; DDB: DuckDB.**

| | float input | | | | integer input | | |
|---|---|---|---|---|---|---|---|
| Input | Z | PQ | ALP | **MorletDB** | FLAC | PQ | DDB |
| CX counts | 8.33 | 6.84 | 9.50 | **4.61** | 4.38 | 5.24 | 6.38 |
| IMS 3-decimal | 13.24 | 42.94 | 11.00 | **8.81** | 8.41 | 9.87 | 10.88 |
| Paderborn counts | 12.04 | 9.48 | 10.75 | **7.88** | 7.70 | 9.61 | 11.25 |
| Paderborn physical | 14.82 | 30.57 | 62.13 | **9.17** | 7.70 | 9.61 | 11.25 |
| Sleep-EDF EEG | 23.54 | 38.15 | 59.63 | **15.51** | 8.65 | 10.22 | 12.00 |
| PTB-XL ECG | 18.94 | 44.64 | 12.75 | **7.40** | 7.46 | 10.27 | 12.75 |

### 2.2 Codec against modern baselines (B1)

B1 compares codecs on identical samples, each verified lossless [17]. Float codecs receive only float64 values: zstd level 19 per chunk, Parquet with `BYTE_STREAM_SPLIT` and zstd, and DuckDB 2.0's automatic float compression, which uses ALP and its fallback ALP-RD [1, 7]. MorletDB receives the values plus the nominal quantum. Integer codecs receive the exact digitizer counts: FLAC [3], Parquet delta coding with zstd, and DuckDB bit-packing. Two inputs extend the earlier storage inputs: a Sleep-EDF EEG slice [8, 9] from 32 nights with symmetric and asymmetric calibration ranges (an earlier engineering check had measured MorletDB's own storage of this collection, not the other arms), and PTB-XL ECG lead I [10], new to the storage studies. The Parquet float arm is one fixed encoding, not a tuned Parquet baseline; ALP is DuckDB 2.0.0's implementation, measured as table blocks.

Figure 1 and Table 2 show three regimes.

*Counts and decimals.* When values are integers or short decimals (CX, IMS, Paderborn counts, ECG with gain 1000), ALP and MorletDB both recover integers; MorletDB is 20–51% smaller than ALP, plausibly because consecutive samples are correlated and prediction exploits that, while ALP's frame-of-reference bit-packing

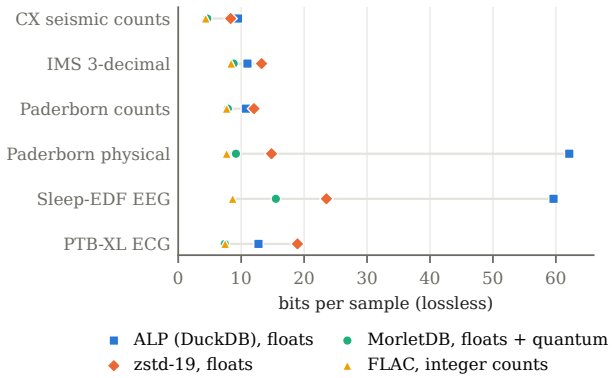



**Figure 1: B1: lossless bits per sample on identical samples. Float codecs see only float64 values; MorletDB also sees the nominal quantum; FLAC sees integer counts. Table 2 gives all arms.**

does not; B1 does not isolate this. MorletDB is within 5% of FLAC, whose LPC stage it lacks, and slightly below FLAC on ECG.

*Calibrated floats.* On Paderborn physical values and Sleep-EDF, no decimal exponent turns values into integers, and the tested DuckDB ALP implementation selected ALP-RD, storing about 60 bits per sample, near raw. Of the three metadata-free arms, zstd was smallest. MorletDB, which additionally receives the nominal quantum, was 1.5–1.6× smaller than zstd. Our pre-registered claim of at least 2× was not met. B2 added ClickHouse (26.5, best of four float codec chains) and TileDB (2.30, best of four filter pipelines) on the same samples [23]: on the calibrated inputs MorletDB was 0.66 and 0.76× the best of them (ClickHouse `ZSTD(19)` on both), so it remained the smallest metadata-free arm; FLAC remained the best integer codec on every input.

*The correction gap.* Given counts, MorletDB codes the two calibrated inputs at 7.9 and 8.8 bits per sample; given the calibrated floats, it needs 1.3 and 6.7 bits per sample more. That extra cost, relative to MorletDB's own integer-input path, is the correction stream: the source software computed $x$ with a particular sequence of roundings (on Sleep-EDF a multiplication, a division and an addition), which a single fitted step and offset does not reproduce for every sample. It does not explain the whole difference from FLAC, which also includes MorletDB's integer path being 0.15–0.18 bits per sample larger than FLAC's. Replaying the source's calibration arithmetic might remove the corrections when the correct parameters and operation sequence are known; this is neither implemented nor tested. Our pre-registered claim that MorletDB stays within 15% of the best integer codec held on four of six inputs and failed on these two.

For float compression, these inputs illustrate that real-valued sensor data can be neither decimal nor arbitrary but an affine image of integers under floating-point rounding. On the two such inputs we tested, the decimal-oriented ALP implementation fell back to near-raw size and the best byte-oriented arm was 1.5–1.6× larger than coding with a declared quantum.

**Table 3: R1b on 2022 windows. Size relative to the raw chunk table; latency includes the client path.**

| Envelopes | Size/raw | Recall@10 | Same order | p95 ms |
|---|---|---|---|---|
| from raw (reference) | — | 1 | 1 | 1324 |
| float32 | 28.1% | 1.000 | 1.000 | 14 |
| **float16** | **12.9%** | **0.9999** | **0.987** | **16** |
| uint8 | 7.2% | 0.998 | 0.845 | 17 |
| uint8, coarser | 4.2% | 0.952 | 0.067 | 16 |

# 3 ENVELOPE PRECISION AND ZERO-LAG STORAGE (R1B, P1)

## 3.1 Precision of materialized envelopes (R1b)

R1b selected an envelope precision on 2023 seismic windows and evaluated it once on 2022 event and noise windows that, according to its report, no earlier study had read: 1,826 windows at ten CX stations [11], 1,557 event windows from 284 catalogued events and 269 no-match distractors [18]. The reference decodes raw samples and computes envelopes in the client.

Float16 envelopes met all frozen criteria (Table 3): recall@10 of 0.9999, 12.9% of the raw table, and an unchanged catalogue-distance relevance proxy (+0.001 km, 95% interval [−0.002, +0.004]). The roughly 80-fold latency reduction measures materialization against a client that decodes and recomputes; it is not a speedup over an optimized raw-scanning engine.

## 3.2 Same storage, different query (P1)

P1 stored the identical float16 envelopes in a pgvector `halfvec(410)` column [6]. Both tables occupied exactly 1,794,048 bytes, and all 1,826 envelopes were bit-identical, so envelope storage itself is no advantage. Zero-lag cosine search, however, shared only 64.7% of the lag-tolerant top ten and reproduced its order for 0.1% of queries. Scoring lags in the client on the same `halfvec` rows reproduced MorletDB's ranking exactly. In a later same-run comparison, median latency was 6.85 ms for MorletDB, 0.49 ms for zero-lag cosine and 20.3 ms for client lag scoring. The first two answer different questions, so their ratio is not a like-for-like speed comparison.

**Representation overhead (R2 [19]).** The materialized envelopes took 12.9% of the raw chunk table in R1b but 60.2% in R2. Measured sizes account for the factor of 4.68: halving the stride from 4096 (one envelope per event window) to 2048 doubles the number of envelopes per sample (×2.0); R2's continuous, mostly ambient recordings compress to 5.04 payload bits per sample against 12.39 for R1b's event windows (×2.46); row and TOAST overheads differ slightly (×0.95 combined). The ratio thus depends on stride, raw compressibility and physical layout, and the 12.9% figure does not transfer to continuous archives. R2's benchmark held four structures side by side: raw chunks (371 MB, always required), envelopes (223 MB, used by the exhaustive scan and every re-rank), the `halfvec` copy (241 MB) and its HNSW index (275 MB). The copy stores the same values as the envelopes, so a deployment would keep one of the two if its scorer read that format (MorletDB's reads its own); the index is needed only for an index-based shortlist.

## 4 BUILDING REPRESENTATIONS (CW1)

As a first step toward compaction-style workers that run representation work outside the database, CW1 compared building envelopes inside PostgreSQL with stateless external processes that read encoded chunks through SQL, compute with the same Rust code, and write rows back with key-idempotent inserts [25]. On 33,736 windows the external path produced byte-identical rows in every run, survived a killed and restarted worker and two workers processing the same recordings, and moved 90% of the per-window CPU off the database server (0.13 against 1.31 ms of database CPU per window). It also used about four times less CPU in total, which exposed the real inefficiency: the in-database builder issued one SQL statement per window and decoded every raw sample twice. Batching the in-database builder, so that each chunk is decoded once for up to 256 windows, reduced database CPU to 0.31 ms per window and raised one session from 740 to 3,084 windows per second (four sessions: 12,034), byte-identical to before. External workers remain a viable basis for separating compute from storage, but on one machine batching delivered most of their benefit without them.

## 5 ACCESS CONTROL, RECOVERY AND LOAD

Before a bounded pilot with human review of results, we tested access control, recovery and sustained load against plans frozen before each run [24]. All runs shared the machine with an unrelated CPU-intensive job (load average 3–20), so latencies are indicative; correctness results are not affected.

**Access control.** With row-level security only on the recordings table, a tenant could still read other tenants' chunks, envelopes and probe vectors by querying the companion tables directly: search functions honoured the policy, the tables did not. `morlet\_enable\_row\_security` now applies a policy to every companion table that admits rows whose recording is visible. Its first form, a correlated `EXISTS`, prevented parallel plans and tripled exact search time (946 to 2,821 ms on 235,000 windows); comparing against an array of visible recording identifiers kept the parallel scan (995 ms, +5%) and cost probe search 6 ms. We did not test policies with thousands of visible recordings.

**Recovery.** On a 50,604-window database every check passed: each recording hashed identically afterwards, each complete window had exactly one envelope and probe row, and exact searches returned identical lists. A full dump restored in 18 s; dumping only recordings and chunks took a third of the space (146 against 459 MB), and rebuilding envelopes and the probe index after restore took 23 s. Twenty backends terminated at random points inside append transactions left no partial rows, and replaying the chunk succeeded; aborted builds resumed, and an aborted probe enable could be retried. Killed and duplicated external workers produced exactly one byte-identical row per window, but did not write probe rows, so their windows stayed invisible to probe search until filled. After deleting 15% of recordings, `VACUUM` took 113 s and left the index size unchanged, whereas an online `REINDEX CONCURRENTLY` took 10 s while four clients searched (p95 23 to 31 ms). An extension update forced to fail midway rolled back completely.

**Replay.** Replaying one day of four development channels (16,868 windows) with the builder called in each append transaction, every window was searchable at its commit while appends ran at 919,000 samples/s, 53% of raw-only throughput. At 200× real time, probe queries during ingestion had a p95 of 82 ms against 189 ms for exact search, at recall 0.9997 on the same snapshot [22].

**Sustained load.** On an archive of 938,768 windows (4.8 GB), sixteen channels at 25.6 kHz (409,600 samples/s) were appended by four writers, each building envelopes and probe rows in the append transaction, while four analysts issued one query every two seconds, half filtered to about 2% of windows and half unfiltered. With PostgreSQL's default configuration the run met its criteria (filtered p95 167 ms; probe p95 337 ms, spot-check recall 0.998; ingest lag at most 22 s, recovered), but wrote 13.8 GB of write-ahead log for 0.76 GB of database growth in 15 minutes: HNSW insertions touch pages across a 1.2 GB index, and each checkpoint forces full-page images of them. At twice the rate the writers fell behind without recovering; capacity was about 1.3–1.4× the target. With a pilot configuration (8 GB shared buffers, 16 GB `max\_wal\_size`, 15-minute checkpoints, WAL compression, changed together) the same run wrote 1.91 GB of WAL for 0.65 GB of growth, kept ingest lag at or below 2 s, and gave filtered and probe p95 of 81 and 100 ms (recall 0.994); background load was lower in this run (7.3 against 11.7), which confounds the latency comparison but not the WAL volume. Unfiltered exact search over the full archive stayed at 7.8–9.7 s p95, which is why the router sends large galleries to probes. A run planned for two hours with the same configuration met the target for 111 minutes (409,585 samples/s; writer lag at most 24 s in the first hour, with the unrelated job running, and 3.4 s in the second; filtered and probe p95 118 and 138 ms, recall 0.993; no errors; 12.0 GB of WAL for 4.7 GB of growth). Then all client processes paused for 90–106 s; the server log shows no error, which does not rule out a system-side cause; and the harness's lag rule, stricter than the protocol's wording, stopped the run at 112.5 minutes; the stored result divides the ingest rate by the scheduled duration, and a re-analysis from the raw logs reports the rate over the elapsed time. The two-hour run therefore did not complete. A measured stability duration is not a production qualification.

## 6 STUDY OVERVIEW

Confirmatory studies committed their protocol (questions, data, arms, metrics, decision rule) to the repository before their first result; exploratory diagnostics fixed a protocol without a confirmatory decision, or had none (pruning); the grid analysis and the choice of illustrative cases are post hoc. Protocols are internal plans with commit times set by the author's machine, not registrations with a third party. E1 re-measured data already used for BL1-C's confirmation and is not another independent confirmation. The full table with questions, results and limitations is `research/paper/STUDIES.md`.

## 7 APPLICATION EVIDENCE AND NEGATIVE RESULTS

**Sleep staging by example (S1).** On 153 Sleep-EDF nights with a subject-disjoint split, staging each 30-second epoch by the 40 most

| Study | Question | Protocol | Results | Data | Status |
|---|---|---|---|---|---|
| R1b | float16 envelopes precise enough? | bfdf6fe 10-26 17:41 | db63d58 19:16 | unread 2022 slice | confirmation |
| R2 | exact scaling; zero-lag first stage? | 12cad82 10-31 14:20 | c07396e 14:55 | 99.9% unread | corrected by R2-D |
| R2-D | ANN error or candidate metric? | 7a7b276 15:57 | 246c91a 16:24 | R2's | exploratory; reuse |
| BL1 | shifted probes, configuration | a5381d7 16:44 | e831e51 17:29 | R2's | development |
| BL1-C | frozen configuration on unread data | cc99d09 17:31 | 635ebd4 18:01 | unread stations, year | confirmation |
| BL1-S | maintenance at commit | ceb63ad 17:33 | 0d5df8b 18:19 | R2's | development |
| SS1 | one stored copy enough? | 1942b83 19:20 | 6a34b63 20:03 | R2's, BL1-C's | reuse; bounds missed |
| SS3 | filters and routing | 71fb9ae 20:21 | 136c81a 20:54 | R2's, BL1-C's | reuse; corrected (SS3b) |
| Pruning | exact pruning $\geq 2\times$? | — | 9aea409 10-30 | 2022 slice | exploratory; stopped |
| Grid analysis | why spacing five? | — | 0ef13d1 11-01 | dev | post hoc |
| C2, C3 | target ingest; 2× overload | 5ad1f12 11-01 09:32 | 5fadadb 10:56 | both | C3 failed |
| C2p, C4 | tuned configuration; 2 hours | 2412a77 10:57 | 167feb6 (#76) | both | C4 aborted |
| E1 | all arms, one build | 34904f6 12:12 | 9c2b6b0 13:26 | dev, BL1-C's | re-measurement |
| D1, D2 | time per stage; correlated filters | fa367ed 13:02 | e335217 13:32 | dev | exploratory |

similar expert-scored epochs from other subjects gave a mean per-subject Cohen's $\kappa$ of 0.583, against 0.541 for logistic regression on the same features (+0.042, 95% CI [0.016, 0.068]), with equal macro-F1 [20]. Retrieval thus exposes source-linked neighbours at no measured accuracy cost against this simple comparator; part of the $\kappa$ difference comes from the regression's class-balanced weights. The result is far from state-of-the-art staging and makes no clinical claim.

**Morlet versus FFT.** In the same study, FFT band powers with the same band layout gave $\kappa$ 0.589 against 0.583 for Morlet energies. We found no advantage for the wavelet representation and made the extractor replaceable.

**Wavelet and progressive storage.** A compression benchmark found the tested wavelet decomposition codec 19–57% *larger* than scalar quantization with conventional coding at matched error bounds, and progressive raw storage missed its byte-saving gate (0.59 against $\leq 0.5$ of a cascade) because a separate envelope index dominated [21]. This motivated storing a retrieval representation *next to* losslessly coded raw data rather than a multiresolution raw format.

**Reference coding.** Coding a machining recording as a residual against an earlier execution of the same operation would have saved at most 0.0019% on the Bosch CNC data [12], because executions were too dissimilar sample by sample [26].

**Out of scope.** Three later studies tested retrieval-based forecasting of bearing and turbofan degradation and current-to-vibration estimation; none passed its gate [15]. They concern analytics on top of the store rather than storage and retrieval.

**Licence.** © 2026 Zin Nam. Text and figures: CC BY 4.0 (https://creativecommons.org/licenses/by/4.0/); source code: Apache-2.0.

## REFERENCES


[1] Azim Afroozeh, Leonardo X. Kuffo, and Peter Boncz. 2023. ALP: Adaptive Lossless floating-Point Compression. *Proc. ACM Manag. Data* 1, 4, Article 230 (Dec. 2023), 26 pages. doi:10.1145/3626717.

[2] Tony Robinson. 1994. SHORTEN: Simple lossless and near-lossless waveform compression. Technical Report CUED/F-INFENG/TR.156, Cambridge University Engineering Department.

[3] Martijn van Beurden and Andrew Weaver. 2024. Free Lossless Audio Codec (FLAC). RFC 9639, RFC Editor. doi:10.17487/RFC9639.

[4] Robert F. Rice. 1979. Some Practical Universal Noiseless Coding Techniques. JPL Publication 79-22 (NASA-CR-158515), Jet Propulsion Laboratory.

[5] Bob Kemp, Alpo Värri, Agostinho C. Rosa, Kim D. Nielsen, and John Gade. 1992. A simple format for exchange of digitized polygraphic recordings. *Electroencephalography and Clinical Neurophysiology* 82, 5, 391–393. doi:10.1016/0013-4694(92)90009-7.

[6] pgvector contributors. pgvector: open-source vector similarity search for Postgres, version 0.8.0. https://github.com/pgvector/pgvector.

[7] Mark Raasveldt and Hannes Mühleisen. 2019. DuckDB: an Embeddable Analytical Database. In *Proc. 2019 Int. Conf. on Management of Data (SIGMOD '19)*, 1981–1984. doi:10.1145/3299869.3320212.

[8] Bob Kemp, Aeilko H. Zwinderman, Bert Tuk, Hilbert A. C. Kamphuisen, and Josefien J. L. Oberyé. 2000. Analysis of a sleep-dependent neuronal feedback loop: the slow-wave microcontinuity of the EEG. *IEEE Trans. Biomed. Eng.* 47, 9, 1185–1194. doi:10.1109/10.867928.

[9] Ary L. Goldberger, Luis A. N. Amaral, Leon Glass, Jeffrey M. Hausdorff, Plamen Ch. Ivanov, Roger G. Mark, Joseph E. Mietus, George B. Moody, Chung-Kang Peng, and H. Eugene Stanley. 2000. PhysioBank, PhysioToolkit, and PhysioNet: Components of a New Research Resource for Complex Physiologic Signals. *Circulation* 101, 23, e215–e220. doi:10.1161/01.CIR.101.23.e215.

[10] Patrick Wagner, Nils Strodthoff, Ralf-Dieter Bousseljot, Dieter Kreiseler, Fatima I. Lunze, Wojciech Samek, and Tobias Schaeffter. 2020. PTB-XL, a large publicly available electrocardiography dataset. *Scientific Data* 7, Article 154. doi:10.1038/s41597-020-0495-6.

[11] GFZ German Research Centre for Geosciences and Institut des Sciences de l'Univers–Centre National de la Recherche CNRS-INSU. 2006. IPOC Seismic Network. Integrated Plate boundary Observatory Chile. Network code CX. doi:10.14470/PK615318.

[12] Mohamed-Ali Tnani, Michael Feil, and Klaus Diepold. 2022. Smart Data Collection System for Brownfield CNC Milling Machines: A New Benchmark Dataset for Data-Driven Machine Monitoring. *Procedia CIRP* 107, 131–136. doi:10.1016/j.procir.2022.04.022.

[13] Christian Lessmeier, James Kuria Kimotho, Detmar Zimmer, and Walter Sextro. 2016. Condition Monitoring of Bearing Damage in Electromechanical Drive Systems by Using Motor Current Signals of Electric Motors: A Benchmark Data Set for Data-Driven Classification. In *Proc. European Conf. of the PHM Society (PHME 2016)*. doi:10.36001/phme.2016.v3i1.1577.

[14] J. Lee, H. Qiu, G. Yu, J. Lin, and Rexnord Technical Services. 2007. Bearing Data Set. IMS, University of Cincinnati. NASA Prognostics Data Repository, NASA Ames Research Center. https://data.nasa.gov/dataset/ims-bearings.

[15] MorletDB contributors. 2026. MorletDB source repository, protocols and results. https://github.com/merkhet-io/morletdb.

[16] MorletDB contributors. Plain PostgreSQL baselines P1 and P2. `evaluation/plain_postgres/README.md` in [15].

[17] MorletDB contributors. B1: lossless storage against modern baselines. `evaluation/storage_baselines/` in [15].

[18] MorletDB contributors. R1b confirmation (2022). `evaluation/materialized/compact/results/conf-2022/RESULTS.md` in [15].

[19] MorletDB contributors. R2: retrieval scaling, and R2-D diagnostic. `evaluation/retrieval_scaling/` in [15].

[20] MorletDB contributors. S1: sleep staging by retrieval. `evaluation/sleep/RESULTS.md` in [15].

[21] MorletDB contributors. Stage-1 compression benchmark. `evaluation/compression/results/ab70819/RESULTS.md` in [15].

[22] MorletDB contributors. BL1: shifted-probe retrieval, confirmation and ingestion. `evaluation/bounded_lag/` in [15].

[23] MorletDB contributors. B2: ClickHouse and TileDB storage baselines. `evaluation/storage_b2/RESULTS.md` in [15].

[24] MorletDB contributors. Pilot stages 1–2: access control, recovery and load. `evaluation/pilot/RESULTS.md` in [15].
[25] MorletDB contributors. CW1: external compute workers. `evaluation/external_worker/CW1_RESULTS.md` in [15].
[26] MorletDB contributors. X1: reference coding across repeated operations. `evaluation/compression/reference/RESULTS.md` in [15].